



ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO
FACULTAD: INFORMÁTICA Y ELECTRÓNICA
ESCUELA DE INGENIERÍA EN SISTEMAS
CARRERA: SOFTWARE

GUÍA DE LABORATORIO DE APLICACIONES INFORMÁTICAS II

PRÁCTICA No. 2

1. DATOS GENERALES:

NOMBRE DEL ESTUDIANTE

STIVEN APOLO

FECHA DE REALIZACIÓN:

2026/07/03

CODIGO DEL ESTUDIANTE

7243

FECHA DE ENTREGA:

2026/07/03

2. OBJETIVO(S):

2.1.. GENERAL

Aplicar técnicas de identificación, análisis, priorización y clasificación de problemas en la aplicación ClinicFlow, con el fin de establecer acciones de mantenimiento de software que permitan mejorar su funcionamiento, adaptación, usabilidad y mantenibilidad.

2.2.. ESPECÍFICOS

- Identificar los principales problemas técnicos, funcionales y de experiencia de usuario presentes en la aplicación objeto de mantenimiento.
- Analizar el impacto, esfuerzo y riesgo de los problemas detectados para establecer una priorización adecuada.
- Clasificar los problemas según los tipos de mantenimiento correctivo, adaptativo, perfectivo y preventivo.
- Proponer acciones de mejora orientadas a estabilizar la aplicación, facilitar su despliegue y prevenir fallos futuros.

3. METODOLOGÍA

Para el desarrollo de la práctica se aplicó una metodología de análisis de mantenimiento basada en revisión técnica, análisis funcional y priorización por impacto-esfuerzo. El proceso inició con la revisión del comportamiento esperado de ClinicFlow y continuó con la identificación de problemas relacionados con configuración, datos iniciales, comunicación frontend-backend, estado de la interfaz, caché, pruebas y estructura del código.

Posteriormente, cada problema fue evaluado considerando su impacto sobre el sistema, la facilidad de corrección y el tipo de mantenimiento al que pertenece. Finalmente, se organizó una matriz de priorización para determinar qué acciones deben atenderse primero dentro del proceso de mantenimiento.

4. EQUIPOS Y MATERIALES:

- Computador
- Entorno integrado de desarrollo (IDE)
- Aula virtual
- Acceso a internet
- Bibliografía
- Repositorio ClinicFlow
- Node.js, Next.js, Express, MongoDB y Redis
- Herramientas de análisis de código y documentación

5. MARCO TEORICO:

Mantenimiento de software. El mantenimiento de software comprende el conjunto de actividades destinadas a modificar, corregir, adaptar o mejorar una aplicación después de su entrega. Estas actividades permiten conservar la utilidad del sistema, aumentar su vida útil y responder a nuevas necesidades técnicas o funcionales.

Mantenimiento correctivo. Consiste en corregir errores, fallos o defectos detectados durante la ejecución del sistema. Su propósito es restablecer el funcionamiento esperado de la aplicación.

Mantenimiento adaptativo. Permite ajustar el software a cambios del entorno, tales como nuevas versiones de sistemas operativos, librerías, servicios externos, configuraciones de despliegue o requisitos técnicos.

Mantenimiento perfectivo. Busca mejorar características ya existentes del sistema, como rendimiento, usabilidad, calidad visual, claridad de mensajes, eficiencia de procesos o legibilidad del código.

Mantenimiento preventivo. Tiene como objetivo reducir la probabilidad de fallos futuros mediante refactorización, actualización de dependencias, documentación, pruebas automatizadas y mejora de la estructura interna del código.

Priorización de problemas. La priorización permite ordenar los problemas detectados según criterios como impacto, urgencia, esfuerzo y riesgo. Una herramienta común para este análisis es la matriz impacto-esfuerzo, que facilita decidir qué problemas deben resolverse primero.

6. PROCEDIMIENTO:

6.1.. Descripción de la aplicación objeto de mantenimiento

La aplicación seleccionada es ClinicFlow, una plataforma web para la gestión de una clínica y sus citas médicas. El sistema considera diferentes roles de usuario: administrador, recepcionista, doctor y paciente. Su arquitectura se compone de un frontend desarrollado en Next.js, un backend basado en Express, una base de datos MongoDB, Redis para caché y estado temporal, y Socket.IO para notificaciones en tiempo real.

6.2.. Técnicas aplicadas para identificar problemas

Técnica	Descripción	Resultado
Revisión del código fuente	Inspección de archivos de configuración, controladores, rutas, reducers y estructura general del proyecto.	Detección de errores lógicos, problemas de configuración y acoplamiento.
Análisis de despliegue	Revisión del comportamiento de la aplicación en entorno local, Docker y producción.	Identificación de problemas de variables de entorno, CORS y URLs.
Análisis funcional	Observación de flujos como login, departamentos, doctores, reserva de citas y dashboard.	Detección de fallos visibles para el usuario final.
Análisis de experiencia de usuario	Evaluación de estados vacíos, mensajes y feedback de la interfaz.	Propuestas de mejora perfecta.
Matriz impacto-esfuerzo	Evaluación del impacto y dificultad de resolución de cada problema.	Priorización de acciones de mantenimiento.
Clasificación por mantenimiento	Agrupación de problemas según correctivo, adaptativo, perfectivo y preventivo.	Organización técnica de las soluciones propuestas.

6.3.. Identificación de problemas

N.º	Problema	Descripción	Tipo	Impacto	Facilidad
1	La app no trae datos iniciales para demo	El repositorio no incluye datos precargados; vistas como Departments aparecen vacías.	Correctivo / Preventivo	Alto	Alta
2	Variables de entorno incompletas o no alineadas	Faltan o no están bien configuradas variables como CORS_ORIGIN, TOKEN_SECRET, NEXT_PUBLIC_SERVER_URL y MONGODB_URL.	Correctivo / Adaptativo	Alto	Alta
3	Problema de CORS entre frontend y backend	El frontend puede cargar, pero no consumir la API si el backend no permite el origen correcto.	Correctivo / Adaptativo	Alto	Alta
4	Redirecciones y consumo de API dependen mucho	La aplicación se comporta de forma distinta en local, Docker o producción.	Adaptativo	Medio-Alto	Media

	del entorno				
5	Error o inconsistencia en reducers de Redux	La lógica de estado puede actualizar mal elementos, por ejemplo durante la edición de doctores.	Correctivo	Alto	Alta
6	Manejo de caché Redis con datos obsoletos	Redis puede devolver datos antiguos si no se invalidan correctamente las claves.	Correctivo / Preventivo	Medio-Alto	Media
7	Falta de mensajes de error y feedback claros	En varios flujos el usuario no recibe mensajes suficientemente claros cuando algo falla.	Perfectivo	Medio	Alta
8	Falta de mejora visible de UX en estado vacío	Cuando no hay departamentos, la pantalla muestra un vacío simple sin guiar al usuario.	Perfectivo	Medio	Alta
9	Falta de pruebas automatizadas	No hay pruebas para flujos críticos como login, reserva de cita, check-in o dashboard.	Preventivo	Alto	Baja
10	Controladores grandes y lógica mezclada	Existen archivos con demasiada lógica junta, dificultando el mantenimiento futuro.	Preventivo / Perfectivo	Medio-Alto	Baja-Media

6.4.. Análisis de los problemas identificados

Problemas de funcionamiento. Los problemas más críticos son aquellos que impiden o afectan directamente el uso de la aplicación. En este grupo se encuentran la ausencia de datos iniciales, las variables de entorno incompletas, los problemas de CORS y las inconsistencias en Redux. Estos problemas deben atenderse primero porque afectan la validación funcional del sistema.

Problemas de configuración y adaptación. ClinicFlow depende de diferentes entornos de ejecución. Cuando las URLs, variables, redirecciones o políticas CORS no se encuentran alineadas, la aplicación puede fallar en local, Docker o producción. Por ello, estos problemas se relacionan con el mantenimiento adaptativo.

Problemas de experiencia de usuario. La falta de mensajes claros, loaders y estados vacíos informativos no impide completamente el funcionamiento, pero reduce la calidad de uso. Estas mejoras son perfectivas porque hacen que el sistema sea más comprensible para usuarios administrativos, médicos y pacientes.

Problemas de mantenibilidad futura. La falta de pruebas automatizadas y los controladores con lógica mezclada aumentan el riesgo de introducir nuevos errores en cambios posteriores. Estos problemas se tratan como mantenimiento preventivo porque buscan evitar fallos futuros.

6.5.. Priorización de problemas

Prioridad	Problema	Impacto	Esfuerzo	Justificación
1	Variables de entorno incompletas o no alineadas	Alto	Bajo	Sin variables correctas, el sistema puede no conectarse al backend, base de datos o servicios externos.
2	Problema de CORS entre frontend y backend	Alto	Bajo	Impide el consumo de la API desde el frontend y afecta directamente la funcionalidad.
3	La app no trae datos iniciales para demo	Alto	Bajo	Las vistas principales pueden aparecer vacías, dificultando la validación funcional del sistema.
4	Error o inconsistencia en reducers de Redux	Alto	Bajo	Puede provocar errores visibles en la actualización de datos de la interfaz.
5	Manejo de caché Redis con datos obsoletos	Medio-Alto	Medio	Puede mostrar información desactualizada y afectar la confiabilidad del sistema.

6	Redirecciones y consumo de API dependen mucho del entorno	Medio-Alto	Medio	Afecta la portabilidad entre local, Docker y producción.
7	Falta de mensajes de error y feedback claros	Medio	Bajo	Mejora la experiencia del usuario y facilita la comprensión de fallos.
8	Falta de mejora visible de UX en estados vacíos	Medio	Bajo	Es una mejora rápida y visible para demostrar mantenimiento perfecto.
9	Controladores grandes y lógica mezclada	Medio-Alto	Medio-Alto	Dificulta el mantenimiento futuro y requiere refactorización progresiva.
10	Falta de pruebas automatizadas	Alto	Alto	Es fundamental para prevenir errores, pero requiere más tiempo de implementación.

6.6.. Matriz impacto-esfuerzo

Categoría	Problemas incluidos	Acción recomendada
Alto impacto / Bajo esfuerzo	Variables de entorno, CORS, datos iniciales, reducers Redux.	Resolver primero. Son cambios rápidos y mejoran mucho el funcionamiento.
Alto impacto / Alto esfuerzo	Pruebas automatizadas.	Planificar como mejora preventiva importante.
Medio impacto / Bajo esfuerzo	Mensajes de error y estados vacíos.	Implementar como mejoras perfectivas visibles.
Medio-Alto impacto / Medio esfuerzo	Redis y redirecciones por entorno.	Resolver después de estabilizar la configuración base.
Medio-Alto impacto / Alto esfuerzo	Controladores grandes.	Refactorizar progresivamente por módulos o servicios.

6.7.. Clasificación por tipos de mantenimiento

a. Mantenimiento correctivo: Corrección de errores y fallos detectados.

Problema	Corrección propuesta
La app no trae datos iniciales para demo	Crear un script seed para cargar departamentos, doctores, pacientes y citas de ejemplo.
Variables de entorno incompletas	Crear archivos .env.example, .env.local y .env.docker correctamente documentados.
Problema de CORS	Ajustar CORS_ORIGIN para permitir el origen correcto del frontend según el entorno.
Error en reducers de Redux	Revisar los reducers relacionados con edición y actualización de entidades, especialmente doctores.
Caché Redis con datos obsoletos	Invalidar claves después de operaciones de creación, edición o eliminación.

Estas acciones permiten corregir fallos que afectan directamente el funcionamiento de ClinicFlow, especialmente el despliegue, el consumo de la API, la visualización de datos y la consistencia del estado entre frontend, backend y base de datos.

b. Mantenimiento adaptativo: Cambios por nuevas versiones, entornos o configuraciones.

Problema	Adaptación propuesta
Variables de entorno incompletas o no alineadas	Estandarizar variables para local, Docker y producción.

Problema de CORS	Permitir configuración dinámica de orígenes según el ambiente.
Redirecciones y consumo de API dependen del entorno	Separar URLs del frontend y backend mediante variables específicas.
Dependencia de servicios externos	Documentar configuración de correo, Cloudinary, Redis y MongoDB.

Estas mejoras permiten que la aplicación funcione de manera estable en distintos ambientes de ejecución, evitando errores causados por diferencias entre desarrollo local, Docker y producción.

c. Mantenimiento perfectivo: Mejoras de rendimiento, usabilidad o calidad del código.

Problema	Mejora propuesta
Falta de mensajes de error claros	Mostrar mensajes específicos en login, reserva de cita, check-in y dashboard.
Estado vacío poco informativo	Diseñar estados vacíos con mensaje, explicación y acción sugerida.
Controladores grandes y lógica mezclada	Separar parte de la lógica en servicios para mejorar legibilidad.
Feedback visual limitado	Agregar indicadores de carga, confirmaciones y notificaciones más claras.

Estas acciones mejoran la experiencia del usuario y la calidad interna del sistema. Por ejemplo, cuando no existan departamentos, la interfaz debe explicar que no hay registros cargados y sugerir una acción al administrador.

d. Mantenimiento preventivo: Refactorización y actualización de dependencias.

Problema	Acción preventiva
Falta de pruebas automatizadas	Implementar pruebas para login, reserva de cita, check-in y dashboard.
Controladores grandes y lógica mezclada	Refactorizar controladores en servicios, validadores y utilidades.
Redis con datos obsoletos	Definir una estrategia clara de invalidación de caché.
Falta de datos iniciales	Mantener scripts seed reutilizables para pruebas y demostraciones.
Configuración de entorno dispersa	Documentar variables y crear ejemplos actualizados.

Proceso de refactorización. El proceso debe realizarse progresivamente. Primero se identifican los controladores con mayor cantidad de lógica. Luego se extraen funciones repetidas o complejas hacia servicios especializados. Después se separan validaciones, consultas a base de datos, manejo de Redis y respuestas HTTP. Finalmente se ejecutan pruebas para confirmar que el comportamiento funcional no cambió.

Proceso de actualización de dependencias. La actualización debe realizarse de manera controlada. Primero se revisan las versiones actuales de las librerías del frontend y backend. Luego se identifican paquetes desactualizados o con vulnerabilidades. Después se actualizan en una rama separada, se ejecuta la aplicación y se prueban los flujos principales. Finalmente se documentan los cambios y cualquier incompatibilidad encontrada.

6.8.. Plan de mejora propuesto

Fase	Actividades	Resultado esperado
------	-------------	--------------------

Fase 1: Estabilización	Corregir .env, CORS y conexión frontend-backend.	Aplicación funcional en local y Docker.
Fase 2: Datos iniciales	Crear script seed con departamentos, doctores y usuarios demo.	Vistas principales con información visible.
Fase 3: Corrección funcional	Revisar reducers Redux y estados de actualización.	Interfaz con datos consistentes.
Fase 4: Mejora de UX	Agregar mensajes claros, loaders y estados vacíos.	Mejor experiencia para pacientes, doctores y administradores.
Fase 5: Caché y rendimiento	Revisar claves Redis e invalidación.	Menor riesgo de datos obsoletos.
Fase 6: Prevención	Agregar pruebas automatizadas y refactorizar controladores.	Mayor facilidad de mantenimiento futuro.

6.9.. Problemas seleccionados para implementar primero

Orden	Problema	Motivo
1	Variables de entorno incompletas	Es fundamental para que la aplicación funcione correctamente.
2	CORS frontend-backend	Es un error común y fácil de demostrar.
3	Datos iniciales para demo	Permite mostrar la aplicación con información realista.
4	Estado vacío de departamentos	Es una mejora visible de experiencia de usuario.
5	Mensajes de error claros	Mejora la calidad de uso y evidencia mantenimiento perfecto.

Estos problemas son adecuados para la práctica porque permiten demostrar mantenimiento correctivo, adaptativo, perfecto y preventivo sin modificar excesivamente la arquitectura del sistema.

7. CONCLUSIONES Y RECOMENDACIONES:

Conclusiones

- Mediante la revisión técnica y funcional de ClinicFlow se identificaron diez problemas relevantes que afectan la configuración, despliegue, usabilidad y mantenibilidad del sistema.
- Los problemas de mayor prioridad están relacionados con variables de entorno, CORS, ausencia de datos iniciales y errores en el manejo del estado de la interfaz.
- La clasificación por tipos de mantenimiento permitió relacionar cada problema con acciones correctivas, adaptativas, perfectivas o preventivas.
- La matriz impacto-esfuerzo permitió seleccionar las mejoras más convenientes para una primera fase de mantenimiento, priorizando cambios de alto impacto y bajo esfuerzo.

Recomendaciones

- Implementar primero la estandarización de variables de entorno y la corrección de CORS, debido a que ambos aspectos afectan la comunicación entre frontend y backend.

- Crear un script de datos iniciales para facilitar demostraciones, pruebas funcionales y validación de las vistas principales.
- Mejorar los mensajes de error y estados vacíos para que el usuario comprenda mejor el comportamiento de la aplicación.
- Planificar la incorporación progresiva de pruebas automatizadas y refactorización de controladores para prevenir errores en futuras modificaciones.

8. BIBLIOGRAFÍA:

- ClinicFlow. Repositorio del proyecto. GitHub: <https://github.com/Foridul35962/ClinicFlow>
- Sommerville, I. (2011). Ingeniería de software. Pearson Educación.
- Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico. McGraw-Hill.
- IEEE. (2014). ISO/IEC/IEEE 14764: Software Engineering - Software Life Cycle Processes - Maintenance.